

Design Notes

Detailed reasoning behind the design decisions in the README

This document is the deeper version of the README. The README answers what I built, how I built it, and what I'd do with more time, in short form, as the assignment asked. This is the rest of the story for anyone who wants it: the tradeoffs I weighed, the alternatives I rejected, the bug I shipped to myself before catching it, and the runtime reasoning behind each major decision. Nothing here contradicts the README; this just shows the thinking underneath it.

1. Integer cents, not floats

I represent every price as an int in cents (1-99), never a float. Floating-point prices drift - $0.1 + 0.2$ is not 0.3 in floating point - and my engine's entire job is to *compare* prices to decide whether they cross. Drift in a comparison means a wrong match. Integer cents make every comparison exact. The 1-99 bound is the binary market itself: a still-tradeable probability can't be 0 or 100.

```
Price = Annotated[int, Field(ge=1, le=99)] # integer cents,
binary-market bounds
Quantity = Annotated[int, Field(gt=0)] # strictly positive
```

What this cost me: nothing meaningful for a binary market. A market with fractional pricing would need a fixed-point or decimal type instead; here int is both correct and the simplest thing that works.

2. A counter, not a clock - the timing decision

Time priority needs a tiebreaker: when two orders sit at the same price, the earlier one fills first. The obvious instinct is to timestamp each order. I deliberately did not do that. Every order gets a `sequence_number`, a monotonic integer counter incremented once per accepted order, never a wall-clock time.

A wall-clock timestamp fails in two ways. **Collisions:** two orders can arrive inside the same clock resolution and get identical timestamps, which makes FIFO ordering ambiguous - and breaking ties deterministically is the entire point of the `sequence` field. **The clock can run backwards:** NTP corrections and leap-second handling can move the system clock backwards, so a later order could silently receive an earlier timestamp and jump the queue ahead of an order that genuinely came first. In an exchange that isn't a glitch, it's broken fairness, and it's the kind of bug that only shows up in production and is nearly impossible to reproduce.

A counter sidesteps both. It is strictly increasing by construction, so "earlier" is always unambiguous and can never reverse.

I also derive `order_id` straight from this counter (`f"order-{sequence_number}"`) instead of generating a UUID, since the engine already needs a unique, monotonic value and minting a second independent identifier would be redundant.

What this cost me: the counter is per-process state, so it only orders correctly within one engine instance. If I sharded the book across processes, each shard would need its own central sequencer for global cross-shard ordering. For a single-book toy that's irrelevant; in a sharded production system it's the next thing I'd have to design.

3. The data structure, and its runtime

Each side of the book is a `dict[int, deque[Order]]` mapping price level to a FIFO queue, plus one flat dictionary indexing order id to order for cancellation.

```
_bids: { 60: deque([Alice(100), Bob(50)]), 59: deque([Dana(30)]) }
_asks: { 62: deque([Eve(40)]), 65: deque([Frank(80)]) }
_index: { "order-1": Alice, "order-2": Bob, ... }
```

I chose each piece for a runtime reason. **The price-level dict**, with prices sorted on demand via `sorted()`, gives me the best bid and ask without maintaining a separate sorted structure. **The deque within each level** gives $O(1)$ append at the back (a newly resting order) and $O(1)$ pop from the front (the next order to fill) – exactly the FIFO discipline time priority needs. **The flat index dict** turns cancellation into an $O(1)$ lookup; without it, finding an order buried in a level's queue would mean scanning every level and queue, which is $O(N)$.

Because prices are bounded to 1-99, the price-level dict never holds more than 99 keys, so sorting it on every match is sorting at most 99 items – effectively free regardless of how many orders flow through the book. That's also why I didn't reach for a maintained sorted structure (like a `SortedDict`): the thing it would optimize is already capped to a small constant by the domain.

The one place this structure is not $O(1)$, and the most interesting runtime tradeoff in the design, is cancellation within a level. The index makes *finding* a cancelled order $O(1)$, but *removing* it from its level's deque is $O(k)$ in that level's depth, because a deque doesn't support $O(1)$ removal of an interior element – it has to scan to find it. I weighed two alternatives against the deque:

- **Intrusive doubly-linked list per level:** store the list node in the index so a cancel can splice the order out in true $O(1)$. This is what a latency-sensitive production engine would use – but it's more code, more pointers to keep consistent, and a new failure mode: a node unlinked from its list but left in the index becomes a dangling reference, and the next cancel splices into a corrupted structure.
- **Lazy/tombstone deletion:** mark the order cancelled in the index and skip it when the matching loop reaches it. Cancel becomes $O(1)$, but the book carries dead orders until a sweep removes them, so memory grows with cancellations.

For a toy where clarity and provable correctness matter more than cancel latency, the deque is the right call, and I documented the tradeoff directly in a comment in `book.py` rather than silently

accepting it. I worked out the full linked-list design as a “more time” item (see the final section) but didn’t implement it the night before submission, once I saw it touches five different parts of the book.

4. Single-threaded vs. multi-threaded - the decision I changed my mind on

This is the decision I went back and forth on the most, and the final design is stronger because I got it wrong first and had to correct it.

Step one: I worked out that the matching loop itself has to be single-threaded. FIFO only means anything if one thread at a time walks the queue; if two threads were both inside the match loop on the same price level, they’d both see the same resting order and both try to fill it, double-counting the fill.

Step two: I drew the wrong conclusion from that. I reasoned that if the engine is single-threaded by nature, I don’t need a lock, so I removed one I’d added. That felt clean. It was wrong.

Step three: I found the hole, and I reproduced it rather than just suspecting it. FastAPI runs synchronous endpoint handlers in a threadpool, so a submit and a cancel actually land on different threads, and both can end up inside the engine at the same time - one thread mid-match while another mutates the book. I reproduced this as real exceptions (`ValueError: Order not in deque`, `KeyError`) when the two operations overlapped on the same order. My “single-threaded by construction” assumption was an assumption about my code that the web framework quietly broke.

Step four: I weighed two real ways to actually enforce single-threading. **A lock:** wrap every state-mutating method in a `threading.RLock`, so the HTTP server still handles requests concurrently at the OS level but every engine mutation is serialized through the lock. **Constrain the deployment:** run the server with a single worker and a single thread, so only one request is ever in flight. The lock is more code but the guarantee lives in the code itself; the deployment constraint is simpler but fragile, since running it with multiple workers by accident would break it silently with no warning.

I also briefly considered switching to Flask, having read that its development server is single-threaded, and thought that would remove the need for a lock entirely. That turned out to be a trap: Flask’s dev server can’t handle concurrent requests at all, which is fine for local testing, but the moment it runs under a real server like Gunicorn you get multiple workers by default and you’re back in the same problem. It would have hidden the question during development, not answered it. I stayed on FastAPI.

I chose the lock. It’s three lines of code, it survives any deployment configuration, and it says “I enforced this in code, not by hope and documentation.” I used `RLock` rather than `Lock` as insurance against a future self-deadlock if a feature like cancel-replace ever needs one locked method to call another.

The honest tradeoff, both directions: going single-writer buys determinism (the same sequence of orders always produces the same trades, in the same order, which makes the system auditable and replayable), correctness by construction (the cancel-while-matching

race can't exist, because two operations can never be inside the engine at once), and exact FIFO ordering. It costs the ability to use more than one core inside a single book - under heavy load, requests queue behind the lock, and I cannot make one book go faster by adding threads. Multi-threading would buy parallel matching and higher throughput on many cores, at the cost of non-determinism (trade order depending on thread scheduling, which is unacceptable where fairness and replayability matter) and a class of lock-ordering bugs that only appear under load and are miserable to reproduce - exactly the kind I already hit, but worse and intermittent.

My conclusion: for a matching engine, determinism and correctness dominate raw throughput. This isn't unique to a toy - real production engines including CME and major crypto exchanges run their core single-threaded for the same reason. The right way to scale is not to parallelize within a book; it's to keep each book single-writer and deterministic and shard by instrument, so every contract runs on its own single-threaded engine. That's the answer to "how does this scale," and it's the opposite of the instinct to add concurrency.

5. Self-trade prevention - the bug I shipped to myself first

A single owner can legitimately rest a buy and a sell at the same time, as long as they don't cross in price. The problem is only when a new order from an owner would match against that same owner's resting order.

My first implementation used skip-and-continue: on a self-match, skip that resting order and keep looking. I thought it was clean, and it was wrong, which I only found because I probed it instead of trusting that it looked right. After a same-owner submit sequence, the book showed a crossed state - a bid priced above an ask, which is logically impossible. The cause: when there's no other resting order on the opposite side to fall back on, "skip" leaves the incoming order resting on top of the owner's own resting order at a crossing price.

The fix I shipped is to cancel the resting order and let the incoming order proceed - "your newer order supersedes the old one." Cancelling the incoming order instead, or cancelling both, would be equally defensible policies; the requirement is to pick one and hold the invariant that the best bid is always strictly less than the best ask. I check that invariant explicitly with a raise, not an assert, because assert statements are stripped out when Python runs with the -O flag, which would silently disable the most important safety check in the engine exactly where it matters most.

I tested this policy adversarially rather than trusting the fix on inspection: a self-trade order sitting at the front of a FIFO queue with a legitimate order behind it, a self-trade order behind a legitimate one, and multiple own-orders stacked at the same level. All three hold correctly, which I verified by running them, not by reasoning about the code. I'm including this story deliberately, because the skip approach *read* correctly, and the only reason I know it was broken is that I reproduced the crossed book with an actual probe instead of assuming plausible-looking logic was right.

6. Partial fills, and the invariants I protect

quantity is the permanent record of how big an order was; remaining is its live state as it fills. Conflating them would make partial-fill accounting impossible, so they're separate fields, and remaining is the only mutable field on the order. I enforce that with a custom `__setattr__` override – Pydantic's `frozen=True` is all-or-nothing at the model level, which isn't what I needed – so an accidental write to price or side anywhere in the loop raises immediately instead of corrupting the book quietly.

Three invariants hold at all times: remaining is never negative, never exceeds the original quantity, and an order whose remaining hits zero is removed from the book, so no zero-quantity ghost orders are left on a level to "match" for nothing. These guard against the three bug classes I was most worried about: double-counting a fill, an off-by-one on remaining, and a filled husk lingering on the book.

7. Idempotent submission

Every submit carries a client-generated `client_order_id`. The server caches the result per id; a retry with the same id returns the original result instead of placing a duplicate order. This makes the API safe to retry over an unreliable network, where a client genuinely can't tell whether a dropped response means "never arrived" or "arrived, reply lost." I added this beyond what the assignment required, because it's the difference between an API that's safe to put behind a retrying client and one that double-fills on a flaky connection.

The cost I accepted and documented: the cache is unbounded in this toy. Production needs TTL or size-based eviction, accepting that retries older than the eviction window would no longer be deduplicated. I'd rather name that gap than pretend the cache is production-ready.

8. The REST decisions

A few status mappings are deliberate. `POST /orders` returns 201 whether or not the order traded, because the order was created regardless of whether it found a counterparty – the status code answers "was this accepted," and the body answers "what happened to it." `DELETE` collapses "not found" and "wrong owner" into a single 404, deliberately not a 403, because a 403 would tell the caller "this order exists, but it isn't yours," leaking the existence of another participant's order – something a real exchange wouldn't do. The less informative response is the more correct one. And `owner_id` as a query parameter is explicitly a stand-in for authentication, not a substitute for it; it keeps cancel semantics meaningful without building an auth layer that's out of scope for a toy.

What I left out, on purpose

Naming what I left out matters as much as what I built, because it shows the scope was chosen rather than stumbled into. This engine does not implement market orders or order types like IOC/FOK (limit-orders-only by scope), multiple instruments (one book, one contract), persistence across restart (fully in-memory), real authentication, or horizontal scale (single process by design).

The one I want to call out specifically: a dual YES/NO book. In a binary market, buying YES at price p is economically identical to selling NO at $(100 \text{ minus } p)$, and some real prediction-market exchanges run two books and cross YES-buyers against NO-buyers, splitting the dollar of collateral between them. I scoped to a single YES book deliberately, because dual-book crossing is substantial added complexity that isn't needed to demonstrate correct matching. Knowing the duality exists and choosing not to build it is the point.

What I'd do with more time, in full

The README lists my top three. Here is the complete list, in the priority order I'd actually tackle it:

1. **Persistence via an append-only event log.** Record every accepted order and cancel as an event so the book can be deterministically rebuilt by replay. This fits the single-writer model cleanly, since the writer is already the natural serialization point for the log.
2. **Bound the idempotency cache** with TTL or size-based eviction.
3. **Swap the per-level deque for an intrusive doubly-linked list**, so cancellation becomes true $O(1)$ instead of $O(k)$ in that level's depth, for the case I'd actually design for in production: deep books with many orders stacked at a single price. I scoped this out the night before submission once I saw the real surface area - it touches the node structure, the index type (order id to node, rather than order id to order), the FIFO iteration contract the matching loop depends on, and the one place the engine reaches into the book's internals directly. The risk that made me hold it: a node unlinked from its level's list but left in the index is a dangling reference, and the next cancel splices into a corrupted structure - exactly the bug class the deque makes impossible by construction. That's not a risk I wanted to take on hours before a deadline without time to test it properly.
4. **Dual-book YES/NO crossing**, the binary-market-correct version of the book.
5. **A status endpoint** to query a single order's live state by id.
6. **Real authentication** replacing the `owner_id` stand-in.
7. **Scale by sharding per instrument** - keep each book single-threaded and deterministic, and partition contracts across engines, with a central sequencer if global cross-shard ordering is ever needed. Not by adding concurrency inside a book.

How I worked on this

I used Claude as an implementation assistant, to type and pressure-test code against decisions I'd already made, not as a designer. Every choice in this document is mine: the data structure and its runtime tradeoff, the counter-not-clock timing decision, the single-writer concurrency model and the two options I weighed to enforce it, the integer-cent prices, the status-code mappings, and the scoping to a single book. The self-trade and concurrency sections in particular reflect real debugging and a corrected assumption - I shipped skip-and-continue self-trade prevention and a lock-less engine to myself first, reproduced the crossed book and the threadpool race with actual

probes, and fixed both, which is exactly the kind of reasoning a one-shot prompt doesn't produce. The accompanying AI-usage export shows that process directly.